



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251182
Nama Lengkap	Michael Dylan
Minggu ke / Materi	13 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU 1 (40%)

13.3.1 Tuple Immutable

Tuple itu mirip seperti list karena sama-sama bisa menyimpan berbagai jenis data dan memakai indeks integer, tetapi perbedaan utamanya adalah tuple bersifat immutable, artinya isi datanya tidak bisa diubah setelah dibuat. Tuple juga bersifat hashable sehingga bisa digunakan sebagai key pada dictionary Python. Objek hashable sendiri adalah objek yang memiliki nilai hash tetap selama digunakan dan bisa dibandingkan dengan objek lain. Karena nilai hash-nya tetap, objek tersebut dapat dipakai pada struktur data seperti dictionary dan set. Kebanyakan tipe data bawaan Python bersifat hashable, sedangkan tipe data yang bisa diubah seperti list dan dictionary tidak termasuk hashable. Selengkapnya di <https://docs.python.org/3/glossary.html>.

Sintaks tuple:

```
>>> t = 'a','b','c','d','e'
```

Sintaks tuple lainnya menggunakan tanda kurung:

```
>>> t = ('a','b','c','d','e')
```

Tuple dengan satu element dan tambahan koma (,) dibelakang penulisan tuple.

```
>>> t1 = ('a',)
```

```
>>> type(t1)
```

```
<type 'tuple'>
```

Jika tidak ada tambahan koma(,) , akan dianggap sebagai string.

```
>>> t2 = ('a')
```

```
>>> type(t2)
```

```
<type 'str'>
```

Tuple juga bisa dibuat memakai fungsi built-in tuple(), dan jika tidak diberi argumen maka akan menghasilkan tuple kosong.

```
>>> t = tuple()
```

```
>>> print(t)
```

```
()
```

Jika fungsi tuple() diisi data berurutan seperti string, list, atau tuple, maka data tersebut akan diubah menjadi tuple dengan elemen yang berurutan.

('d','u','t','a','w','a','c','a','n','a')

kotak [] dipakai untuk mengakses indeks elemen tuple.

'a'

Untuk tampilkan rentang nilai dari element tuple bisa gunakan:

('b', 'c')

tuple, maka akan didapatkan error.

```
>>> t[0] = 'A'
```

TypeError: object doesn't support item assignment

Element di tuple tidak bisa dirubah tetapi bisa diganti dengan elemen lain.

('A', 'b', 'c', 'd', 'e')

13.3.2 Membandingkan Tuple

```
1 kalimat = 'but soft what light in yonder window breaks'
2 dafkata = kalimat.split()
3 t = list()
4 for kata in dafkata:
5     t.append((len(kata), kata))
6 t.sort(reverse=True)
7 urutan = list()
8 for length, kata in t:
9     urutan.append(kata)
10 print(urutan)
```

Gambar 13.1: Mengurutkan Kata Berdasarkan Panjang. Sumber Python Vscode

Operator perbandingan pada tuple bekerja dengan membandingkan elemen satu per satu mulai dari elemen pertama. Fungsi `sort()` di Python juga bekerja dengan cara yang sama dan teknik ini dikenal sebagai DSU (Decorate, Sort, Undecorate), yaitu membuat tuple sebagai key pengurutan lalu melakukan sorting. Pada program di Gambar 13.1, kalimat dipecah menjadi beberapa kata menggunakan `split()`, lalu setiap kata dibuat menjadi tuple (`panjang_kata`, `kata`) dan dimasukkan ke dalam list. Setelah itu, `sort(reverse=True)` digunakan untuk mengurutkan tuple berdasarkan panjang kata dari yang terbesar ke terkecil. Terakhir, looping kedua mengambil kembali kata dari tuple yang sudah diurutkan sehingga menghasilkan daftar kata yang tersusun dari kata paling panjang hingga paling pendek.

13.3.3 Penugasan Tuple

Python punya fitur tuple di sisi kiri penugasan yang memungkinkan kita mengisi beberapa variabel sekaligus dalam satu baris. Jadi kalau ada list berisi beberapa elemen, kita bisa langsung memisahkan nilai pertama dan kedua ke variabel seperti `x` dan `y` tanpa harus menulis satu per satu. Fitur ini membuat penulisan kode jadi lebih singkat dan rapi karena proses penugasan dilakukan secara berurutan dalam satu statement.

```
>>> m = ['have', 'fun']
>>> x, y = m
>>> x
'have'
>>> y
'fun'
```

Gambar 13.2: Unpacking List ke Variabel dalam Python. Sumber Python Cmd

Python akan menterjemahkan sintaks tuple dengan seperti berikut:

```
>>> m = ['have', 'fun']
>>> x = m[0]
>>> y = m[1]
>>> x
'have'
>>> y
'fun'
```

Gambar 13.3: Terjemahan sintaks tuple. Sumber Python Cmd

Tuple di sisi kiri assignment bisa ditulis tanpa tanda kurung, namun tetap bisa juga menggunakan tanda kurung (parentheses). Keduanya menghasilkan hasil yang sama, yaitu mengisi variabel secara berurutan dari elemen yang ada.

```
>>> m = ['have', 'fun']
>>> (x, y) = m
>>> x
'have'
>>> y
'fun'
```

Gambar 13.4: Tuple menggunakan tanda kurung. Sumber Python Cmd

Tuple bisa digunakan untuk menukar nilai variabel dalam satu statement, misalnya `a, b = b, a`. Pada proses ini, sisi kiri berisi tuple variabel dan sisi kanan berisi tuple ekspresi, lalu setiap nilai di kanan akan dipasangkan ke variabel di kiri. Semua ekspresi di sisi kanan akan dievaluasi dulu sebelum dilakukan penugasan. Hal penting yang harus diperhatikan adalah jumlah variabel harus sama dengan jumlah nilai, jika tidak akan terjadi error seperti “too many values to unpack”. Di sisi kanan assignment biasanya terdapat data sekuensial seperti string, list, atau tuple. Contohnya, kita bisa memisahkan alamat email menjadi username dan domain dengan mudah menggunakan fungsi tertentu.

```
>>> email = 'didanendya@ti.ukdw.ac.id'
>>> username, domain = email.split('@')
>>> print(username)
didanendya
>>> print(domain)
ti.ukdw.ac.id
```

Nilai yang dikembalikan dari split terdiri dua elemen yang dipisahkan tanda “@”, elemen pertama berisi username lalu elemen selanjutnya berisi domain.

13.3.4 Dictionaries and Tuple

Dictionary memiliki metode `items()` untuk mengubah data menjadi list berisi tuple key-value, misalnya dengan `list(d.items())`. Hasilnya berupa pasangan key dan value, namun urutannya mengikuti input, bukan otomatis teratur. Karena sudah menjadi list, data tersebut bisa diurutkan menggunakan `sort()` sehingga menjadi teratur berdasarkan key secara ascending. Dengan cara ini, isi dictionary bisa ditampilkan lebih rapi dan teratur.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> t = list(d.items())
>>> t
[('b', 1), ('a', 10), ('c', 22)]
>>> t.sort()
>>> t
[('a', 10), ('b', 1), ('c', 22)]
```

Gambar 13.5: Mengurutkan Item Dictionary sebagai List Tuples. Sumber Python Cmd

13.3.5 Multipenugasan dengan dictionaries

Kombinasi `items()`, tuple assignment, dan for loop digunakan untuk mengambil key dan value dari dictionary sekaligus dalam satu perulangan. `items()` akan mengembalikan pasangan data dalam

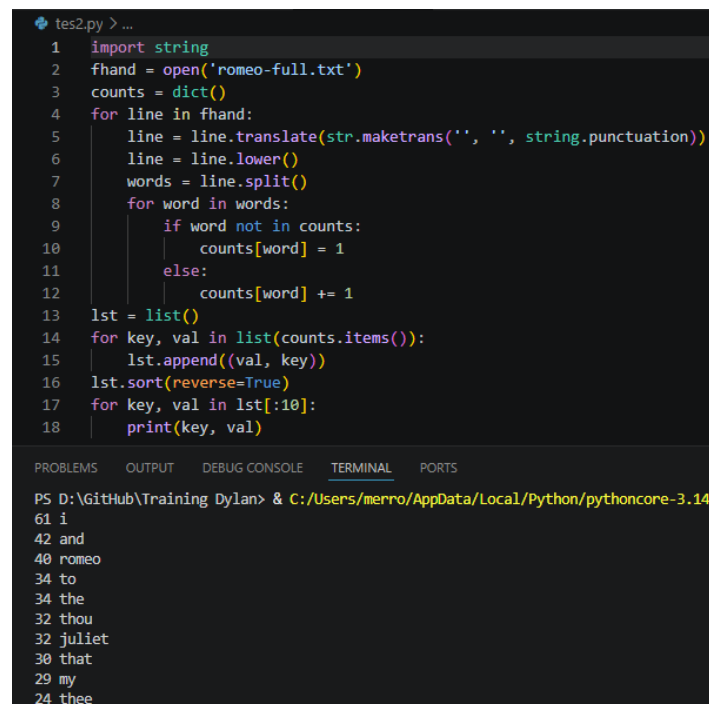
bentuk tuple (key, value), lalu for key, val akan memecah tuple tersebut ke dua variabel. Setiap iterasi akan menampilkan key dan value secara berurutan sesuai isi dictionary. Dengan teknik ini, kita juga bisa mengurutkan atau menampilkan isi dictionary berdasarkan nilai tertentu dari pasangan key-value.

```
>>> d = {'a':10, 'b':1, 'c':22}
>>> l = list()
>>> for key, val in d.items() :
...     l.append( (val, key) ) ...
>>> l
[(10, 'a'), (22, 'c'), (1, 'b')]
>>> l.sort(reverse=True)
>>> l
[(22, 'c'), (10, 'a'), (1, 'b')]
```

Gambar 13.6: Urutan Dictionary Berdasarkan Value Secara Descending. Sumber Python Cmd Langkahnya yaitu membuat list berisi tuple (value, key) dari dictionary menggunakan items(), lalu tiap pasangan dimasukkan ke list. Setelah itu list tersebut di-sort dengan sort(reverse=True) supaya urut berdasarkan value dari besar ke kecil. Dengan cara ini, isi dictionary bisa diurutkan berdasarkan nilai (value) karena value dijadikan elemen pertama dalam tuple.

13.3.6 Kata yang sering muncul

Pada bagian ini, program akan digunakan untuk mencari dan menampilkan 10 kata yang paling sering muncul pada teks *Romeo and Juliet Act 2 Scene 2* dengan memanfaatkan list dan tuple untuk proses pengolahan serta pengurutan datanya.



```
tes2.py > ...
1  import string
2  fhand = open('romeo-full.txt')
3  counts = dict()
4  for line in fhand:
5      line = line.translate(str.maketrans('', '', string.punctuation))
6      line = line.lower()
7      words = line.split()
8      for word in words:
9          if word not in counts:
10             counts[word] = 1
11          else:
12             counts[word] += 1
13  lst = list()
14  for key, val in list(counts.items()):
15      lst.append((val, key))
16  lst.sort(reverse=True)
17  for key, val in lst[:10]:
18      print(key, val)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\GitHub\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14
61 i
42 and
40 romeo
34 to
34 the
32 thou
32 juliet
30 that
29 my
24 thee
```

Gambar 13.7: Menghitung, Menampilkan 10 Kata Paling Sering Muncul . Sumber Python Vscode

Program di atas digunakan untuk menghitung 10 kata yang paling sering muncul pada file Romeo and Juliet. Pertama, program membaca file lalu menghapus tanda baca, mengubah semua huruf menjadi kecil, dan memisahkan tiap kata. Setelah itu, dictionary counts dipakai untuk menghitung jumlah kemunculan setiap kata. Data dari dictionary kemudian diubah menjadi list tuple (value, key) agar bisa diurutkan berdasarkan jumlah kemunculan kata menggunakan sort(reverse=True). Jika ada jumlah yang sama, pengurutan akan mengikuti urutan alfabet dari key. Terakhir, program menampilkan 10 kata dengan frekuensi kemunculan paling tinggi.

13.3.7 Tuple sebagai kunci dictionaries

Tuple bersifat hashable sehingga bisa digunakan sebagai key pada dictionary, sedangkan list tidak bisa. Dengan tuple, kita dapat membuat composite key, misalnya menggabungkan last name dan first name sebagai key untuk menyimpan nomor telepon pada dictionary. Saat melakukan looping, setiap key tuple dapat langsung dipisahkan ke variabel seperti last dan first, lalu digunakan untuk menampilkan nama dan nomor telepon yang sesuai. Seperti di Gambar 13.8.

```
>>> last = 'nendya'
>>> first = 'dida'
>>> number = '088112266'
>>> directory = dict()
>>> directory[last, first] = number
>>> for last, first in directory:
...     print(first, last, directory[last,first])
...
dida nendya 088112266
```

Gambar 13.8: Simpan dan Akses Data Kontak Menggunakan Dictionary. Sumber Python Cmd

BAGIAN 2: LATIHAN MANDIRI (60%)

Link Github = https://github.com/MichaelDylan-ti/71251182_michaeldylan/tree/main

LATIHAN 13.1

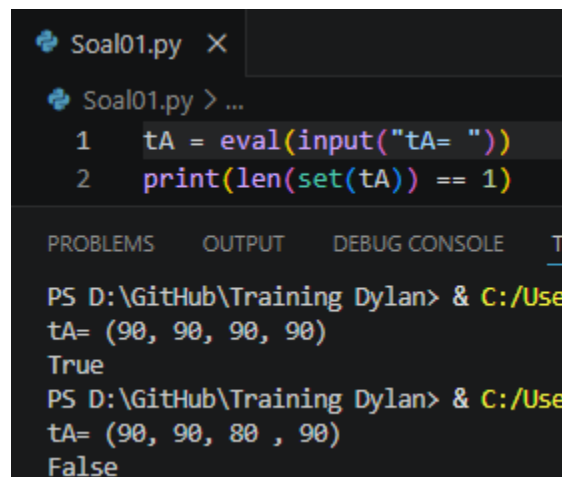
Buatlah program untuk melakukan pengecekan apakah semua anggota yang ada didalam tuple sama. Contoh:

tA= (90, 90, 90, 90)

Output

True

JAWABAN:



```
Soal01.py X
Soal01.py > ...
1 tA = eval(input("tA= "))
2 print(len(set(tA)) == 1)

PROBLEMS OUTPUT DEBUG CONSOLE TE
PS D:\GitHub\Training Dylan> & C:/User
tA= (90, 90, 90, 90)
True
PS D:\GitHub\Training Dylan> & C:/User
tA= (90, 90, 80, 90)
False
```

Gambar 13.9: Input dan Ouput dari Latihan 13.1. Sumber Python Vscode

Program menerima input berupa tuple dari user (misalnya (90, 90, 90, 90)), fungsi eval() mengubah input tersebut menjadi tuple, lalu set(tA) mengubah tuple menjadi himpunan unik sehingga jika semua elemen sama maka panjang set-nya pasti 1, dan outputnya adalah True jika semua anggota tuple sama atau False jika ada yang berbeda.

LATIHAN 13.2

Buatlah program dengan menggunakan tuple yang dapat melakukan proses seperti pada kasus 11.1, Gunakan data diri anda masing-masing dan lakukan perubahan supaya didapatkan output seperti contoh berikut ini :

Contoh:

Data: ('Matahari Bhakti Nendya', '22064091', 'Bantul, DI Yogyakarta')

NIM : 22064091

NAMA : Matahari Bhakti Nendya ALAMAT : Bantul, DI Yogyakarta

NIM: ('2', '2', '0', '6', '4', '0', '9', '1')

NAMA DEPAN: ('a', 't', 'a', 'h', 'a', 'r', 'i')

NAMA TERBALIK: ('Nendya', 'Bhakti', 'Matahari')

JAWABAN:


```
Soal02.py > ...
1 data = ('Michael Dylan', '71251182', 'JL. Kusbisni, Kota Yogyakarta')
2 print("Data: ", data)
3 print()
4
5 nama, nim, alamat = data
6 print("NIM  :", nim)
7 print("NAMA  :", nama)
8 print("ALAMAT:", alamat)
9 print()
10
11 print("NIM:", tuple(nim))
12 print()
13 nama_depan = nama.split()[0]
14 print("NAMA DEPAN:", tuple(nama_depan[1:]))
15 print()
16 nama_terbalik = tuple(nama.split()[::-1])
17 print("NAMA TERBALIK:", nama_terbalik)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\Github\Training Dylan> & C:/Users/merro/AppData/Local/Python/pythoncore-3.14-6
Data: ('Michael Dylan', '71251182', 'JL. Kusbisni, Kota Yogyakarta')

NIM  : 71251182
NAMA  : Michael Dylan
ALAMAT: JL. Kusbisni, Kota Yogyakarta

NIM: ('7', '1', '2', '5', '1', '1', '8', '2')

NAMA DEPAN: ('i', 'c', 'h', 'a', 'e', 'l')

NAMA TERBALIK: ('Dylan', 'Michael')
```

Gambar 13.10: Input dan Ouput dari Latihan 13.2. Sumber Python Vscode

Program ini tidak menerima input dari user karena data sudah langsung didefinisikan dalam tuple data yang berisi nama, NIM, dan alamat, lalu tuple tersebut dicetak utuh sebagai Data, kemudian tuple di-unpack ke variabel nama, nim, dan alamat untuk dicetak satu per satu, selanjutnya nim diubah menjadi tuple karakter per karakter menggunakan tuple(nim), lalu nama depan diambil menggunakan split()[0] dan dicetak karakternya mulai dari indeks ke-1 (huruf kedua) menggunakan slicing [1:] sehingga huruf pertama terbuang, dan terakhir nama lengkap dipisah per kata menggunakan split() lalu dibalik urutannya menggunakan [::-1] dan dijadikan tuple sehingga outputnya adalah nama yang tersusun terbalik dari belakang ke depan.

LATIHAN 13.3

Buatlah program untuk menghitung distribusi jam dalam satu hari dimana ada pesan yang diterima dari setiap email yang masuk. Gunakan file mbox-short.txt untuk sebagai datanya. Berikut ini adalah contoh output dari programmnya.

Contoh: Enter a file name: mbox-short.txt

04 3

06 1

07 1

09 2

10 3

11 6

14 1

15 2

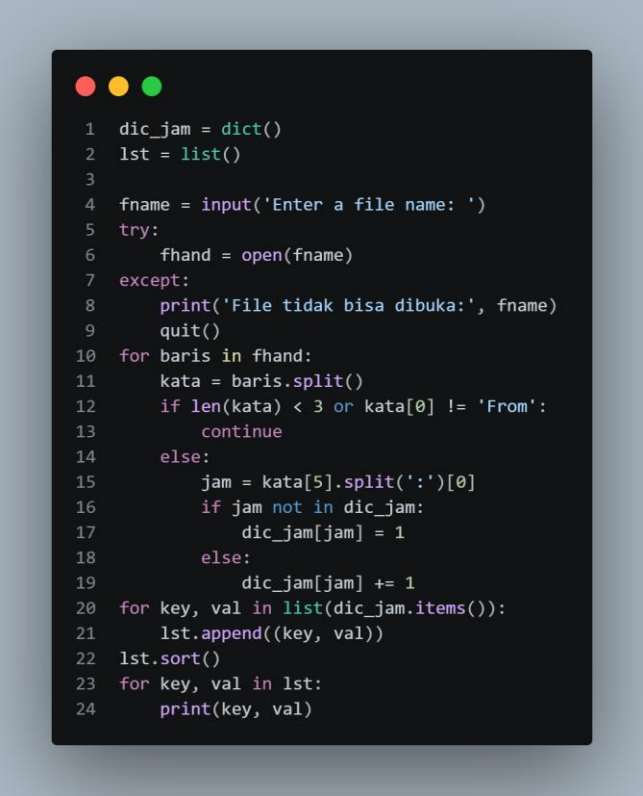
16 4

17 2

18 1

19 1

JAWABAN:



```
1 dic_jam = dict()
2 lst = list()
3
4 fname = input('Enter a file name: ')
5 try:
6     fhand = open(fname)
7 except:
8     print('File tidak bisa dibuka:', fname)
9     quit()
10 for baris in fhand:
11     kata = baris.split()
12     if len(kata) < 3 or kata[0] != 'From':
13         continue
14     else:
15         jam = kata[5].split(':')[0]
16         if jam not in dic_jam:
17             dic_jam[jam] = 1
18         else:
19             dic_jam[jam] += 1
20 for key, val in list(dic_jam.items()):
21     lst.append((key, val))
22 lst.sort()
23 for key, val in lst:
24     print(key, val)
```

Gambar 13.11: Input dari Latihan 13.3. Sumber Python Vscode

```
Enter a file name: mbox-short.txt
04 3
06 1
07 1
09 2
10 3
11 6
14 1
15 2
16 4
17 2
18 1
19 1
```

Gambar 13.12: Output dari Latihan 13.3. Sumber Python Vscod

Program menerima input berupa nama file dari user (contoh: `mbox-short.txt`), lalu membuka dan membaca file tersebut baris per baris, dan hanya memproses baris yang diawali kata “From” sambil melewati baris lainnya menggunakan “continue”, kemudian jam diambil dari elemen ke-5 setiap baris (kolom waktu seperti 09:14:16) menggunakan “split(':')[0]” sehingga hanya angka jamnya saja yang diambil, lalu setiap jam dihitung kemunculannya dan disimpan ke dalam dictionary dic_jam, selanjutnya dictionary dikonversi menjadi list of tuple “(jam, jumlah)” menggunakan “lst.append((key, val))” lalu diurutkan secara ascending berdasarkan jam menggunakan “lst.sort()”, dan outputnya adalah daftar jam beserta jumlah pesan yang diterima pada jam tersebut yang tampil berurutan dari jam terkecil ke terbesar seperti “04 3”, “06 1”, “07 1”, dan seterusnya.